

AWiR – ćwiczenie 7

Temat:

Dodanie SOAP Web Service (server).

Cel ćwiczenia:

Rozszerz aplikację z **Ćwiczeń 4–6** o **serwer SOAP**, który udostępni podobne operacje jak REST API dla User:

- wystaw SOAP Web Service pod adresem /ws (WSDL dostępny dla klientów),
- przygotuj kontrakt (XSD) opisujący typ User i operacje (request/response),
- zaimplementuj endpoint SOAP, który deleguje logikę do istniejącego UserService,
- przetestuj wywołania w narzędziu typu **SoapUI** lub IntelliJ HTTP Client.

REST (/api/**) nadal działa tak jak w ćw. 4–6. SOAP ma być **alternatywnym interfejsem** do tej samej logiki biznesowej.

Niezbędne narzędzia:

1. DE: STS 4 / IntelliJ / VS Code
2. Maven 3.9+
3. JDK 21
4. Spring Boot 3.3.x
5. Narzędzie do testów SOAP: SoapUI / IntelliJ / Postman (z pluginem SOAP)

Dotychczasowy projekt zawierał:

Masz już projekt z ćw. 4–6:

- encja User, UserRepository, UserService,
- kontroler MVC (/users/**),
- REST API (/api/users – CRUD),
- Spring Security (REST = Basic, MVC = formLogin + remember-me),
- logowanie przez Google (ćw. 6).

W tym ćwiczeniu dodaj:

- zależności **Spring Web Services** do pom.xml,
- plik **XSD** z definicją typów i operacji (request/response),
- konfigurację Spring-WS (WebServiceConfig + MessageDispatcherServlet),
- klasę endpointu SOAP (UserEndpoint) mapującą SOAP na UserService,

- prosty test SOAP (np. w SoapUI).

Zmiany w strukturze projektu (pogrubione)

```
src/main/java/edu/zut/awir/
├── AwirApplication.java
├── config/
│   ├── SecurityConfig.java
│   └── **WebServiceConfig.java** # NOWE: konfiguracja Spring-WS + WSDL
├── service/
│   └── UserService.java
├── web/
├── mvc/
│   └── UserController.java
├── rest/
│   ├── UserRestController.java
│   └── RestExceptionHandler.java
├── **soap/**
│   └── **UserEndpoint.java** # NOWE: endpoint SOAP
└──

src/main/resources/
├── application.properties
├── **xsd/users.xsd** # NOWE: kontrakt XSD
├── templates/
│   ├── login.html
│   ├── me.html
│   └── itd.
```

1. Rozszerzenie zależności w pliku pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-web-services</artifactId>
</dependency>
<dependency>
  <groupId>jakarta.xml.bind</groupId>
  <artifactId>jakarta.xml.bind-api</artifactId>
</dependency>
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
</dependency>
```

2. Kontrakt XSD – definicja komunikatów SOAP

```
src/main/resources/xsd/users.xsd
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema"
  targetNamespace="http://zut.edu/awir/users"
  xmlns:tns="http://zut.edu/awir/users"
  elementFormDefault="qualified">

  <xs:element name="getUserRequest">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="id" type="xs:long"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>

  <xs:element name="getUserResponse">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="user" type="tns:user" minOccurs="0"/>
      </xs:sequence>
    </xs:complexType>
  </xs:element>
```

```

</xs:complexType>
</xs:element>

<xs:element name="getUsersRequest" type="xs:anyType"/>

<xs:element name="getUsersResponse">
<xs:complexType>
<xs:sequence>
<xs:element name="users" type="tns:user" minOccurs="0"
maxOccurs="unbounded"/>
</xs:sequence>
</xs:complexType>
</xs:element>

<xs:complexType name="user">
<xs:sequence>
<xs:element name="id" type="xs:long" minOccurs="0"/>
<xs:element name="name" type="xs:string"/>
<xs:element name="email" type="xs:string"/>
</xs:sequence>
</xs:complexType>

</xs:schema>

```

3. Konfiguracja Spring-WS i WSDL

```

src/main/java/edu/zut/awir/config/WebServiceConfig.java
package edu.zut.awir.config;

```

```

import org.springframework.boot.web.servlet.ServletRegistrationBean;
import org.springframework.context.ApplicationContext;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.io.ClassPathResource;
import org.springframework.ws.config.annotation.EnableWs;
import org.springframework.ws.config.annotation.WsConfigurerAdapter;
import org.springframework.ws.transport.http.MessageDispatcherServlet;
import org.springframework.ws.wsdl.wsdl11.DefaultWsdl11Definition;
import org.springframework.xml.xsd.SimpleXsdSchema;
import org.springframework.xml.xsd.XsdSchema;

@EnableWs
@Configuration
public class WebServiceConfig extends WsConfigurerAdapter {

    @Bean
    ServletRegistrationBean<MessageDispatcherServlet>
    messageDispatcherServlet(ApplicationContext ctx) {
        MessageDispatcherServlet servlet = new MessageDispatcherServlet();
        servlet.setApplicationContext(ctx);
        servlet.setTransformWsdlLocations(true);
        return new ServletRegistrationBean<>(servlet, "/ws/*");
    }

    @Bean
    XsdSchema usersSchema() {
        return new SimpleXsdSchema(new ClassPathResource("xsd/users.xsd"));
    }

    @Bean(name = "users")
    DefaultWsdl11Definition defaultWsdl11Definition(XsdSchema usersSchema) {
        DefaultWsdl11Definition wsdl = new DefaultWsdl11Definition();
        wsdl.setPortTypeName("UsersPort");
    }
}

```

```

wsdl.setLocationUri("/ws");
wsdl.setTargetNamespace("http://zut.edu/awir/users");
wsdl.setSchema(usersSchema);
return wsdl;
}
}

```

4. Klasy modelu

Wygeneruj za pomocą maven-jaxb2-plugin na podstawie schematu xsd.

5. Klasa endpointu SOAP

src/main/java/edu/zut/awir/web/soap/UserEndpoint.java

```

package edu.zut.awir.web.soap;

import edu.zut.awir.model.User;
import edu.zut.awir.service.UserService;
// Tu dodaj import klas z modelu
import lombok.RequiredArgsConstructor;
import org.springframework.ws.server.endpoint.annotation.Endpoint;
import org.springframework.ws.server.endpoint.annotation.PayloadRoot;
import org.springframework.ws.server.endpoint.annotation.RequestPayload;
import org.springframework.ws.server.endpoint.annotation.ResponsePayload;

@Endpoint
@RequiredArgsConstructor
public class UserEndpoint {

    private static final String NAMESPACE = "http://zut.edu/awir/users";

    private final UserService userService;

    @PayloadRoot(namespace = NAMESPACE, localPart = "getUserRequest")
    @ResponsePayload
    public GetUserResponse getUser(@RequestPayload GetUserRequest request) {
        GetUserResponse resp = new GetUserResponse();
        User u = userService.findById(request.getId());
        if (u != null) {
            resp.setUser(toUserType(u));
        }
        return resp;
    }

    @PayloadRoot(namespace = NAMESPACE, localPart = "getUsersRequest")
    @ResponsePayload
    public GetUsersResponse getUsers(@RequestPayload GetUsersRequest request) {
        GetUsersResponse resp = new GetUsersResponse();
        userService.findAll().forEach(u -> resp.getUsers().add(toUserType(u)));
        return resp;
    }

    private UserType toUserType(User u) {
        UserType t = new UserType();
        t.setId(u.getId());
        t.setName(u.getName());
        t.setEmail(u.getEmail());
        return t;
    }
}

```

6. Integracja z Security

```

.authorizeHttpRequests(reg -> reg

```

```
.requestMatchers("/ws/**", "/ws").permitAll()  
// reszta reguł...  
)
```

7. Testowanie w SoapUI

Uruchom aplikację.

- Otwórz `http://localhost:8080/ws/users.wsdl` w przeglądarce – sprawdź, czy WSDL się generuje.
- W SoapUI: **New SOAP Project** → podaj URL WSDL.
- Wybierz operację `getUsers`, dodaj i wyślij request.
- Sprawdź odpowiedź – powinna zawierać listę użytkowników z bazy.

8. Zadania do wykonania

- Dodaj zależności Spring-WS i JAXB do projektu.
- Utwórz plik `users.xsd` z definicją typów i operacji (co najmniej: `getUser`, `getUsers`).
- Wygeneruj model za pomocą pluginu `maven-jaxb2-plugin`.
- Skonfiguruj `WebServiceConfig`, aby WSDL było dostępne pod `/ws/users.wsdl`.
- Zaimplementuj `UserEndpoint`, który korzysta z `UserService` i zwraca dane w formacie zgodnym z XSD.
- Przetestuj operacje SOAP w SoapUI lub innym kliencie.